cubst

Draw twisty puzzles in Typst

Version 0.1.0

Contents

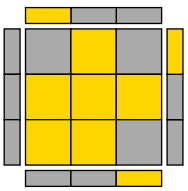
1 Quick start 1
2 Building a state 2
2.1 Notation 3
2.2 Color scheme 4
2.3 Reading a state 5
3 Drawing a state 5
3.1 Views 5
3.2 Mask 7
3.3 Appearance parameters 7
3.3.1 Palette 9
3.3.2 Outlines and gaps 9
3.3.3 Arrows 10
4 Other puzzles 10
4.1 Skewb 10
4.2 Pyraminx 11
4.3 Megaminx 11
4.4 Square-1 12
4.5 Arrows on other puzzles 13
5 Recipes 14
5.1 Algorithm sheet 14
5.2 Step by step 14
5.3 Cross 14
5.4 Other cube sizes 14
6 Reference 15

1 Quick start

```
#import "@preview/cubst:0.1.0": *
```

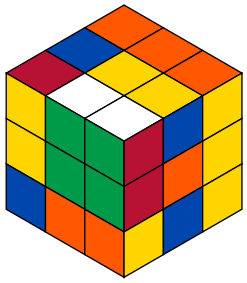
`case()` builds the state an algorithm solves; `draw()` draws it:

```
#let sune = "R U R' U R U2 R'"
#draw(case(sune), view: "oll")
```

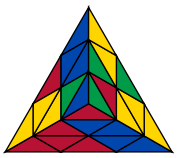


`cube()` builds a scrambled puzzle; `event:` picks the puzzle:

```
#draw(cube(scramble: "R U F2 D' L B'"), view: "full")
```



```
#draw(cube(event: "pyraminx",
  scramble: "U L R' B u"), view: "tip")
```



Everything follows this pattern: build a **state** (Section 2), then draw it (Section 3).

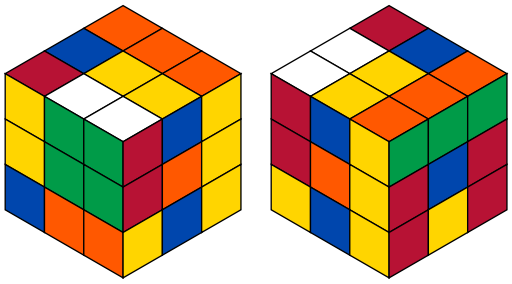
2 Building a state

```
cube(event: "3x3", scramble: none, inverted: false, scheme: auto, options: (:))
case(alg, event: "3x3", scheme: auto, options: (:))
solved(event: "3x3", scheme: auto, options: (:))
apply(c, alg)
```

`event`	which puzzle; case-insensitive, `"3x3x3"`, `"3*3"` and `"3*3*3"` are accepted for `"3x3"`
`scramble`	moves applied to the solved puzzle, in the order written
`inverted`	apply the inverse instead, giving the state that `scramble` **solves**
`scheme`	which color name sits on which face when solved; `auto` = the puzzle's default (Section 2.2)
`options`	settings of one puzzle, currently only the megaminx `cut` (Section 4.3); others are errors

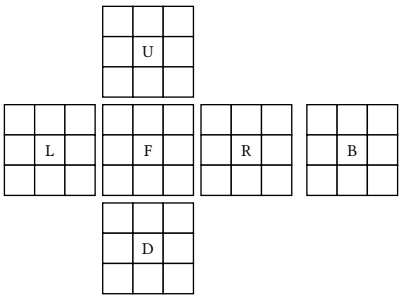
`case(alg)` is `cube(scramble: alg, inverted: true)`: use it for every algorithm diagram (OLL, PLL, …). `apply` adds moves to a state and returns a new one; states are plain values, never modified in place:

```
#let c = cube(scramble: "R U F2 D' L B'")
#draw(c, view: "full")
#draw(apply(c, "y"), view: "full")
```

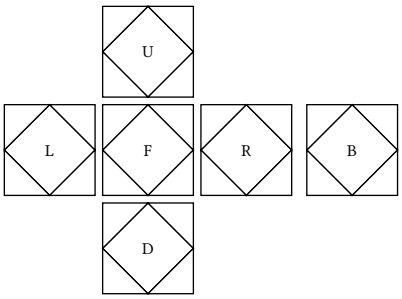


event	**puzzle**	**faces**
`"2x2"`, `"3x3"`, … any `"NxN"`	N×N cube	`U D F B R L`
`"skewb"`	skewb	`U D F B R L`
`"pyraminx"`	pyraminx	`F L R D`
`"megaminx"`	megaminx	`U F R BR BL L D DR DBR B DBL DL`
`"square1"`	Square-1	`U D` (the scheme still names four sides `F B R L` as cubes)

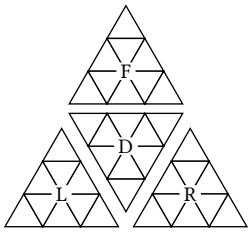
Where the faces are, as the `net` view unfolds them (`labels: "faces"` writes the names). On the Square-1 the piece between the layers is the front of the equator (the middle layer); its `F B R L` sit as on a cube:



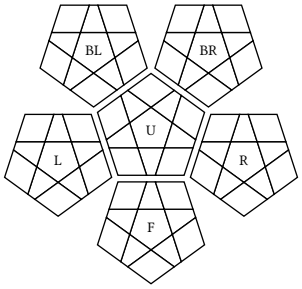
3x3

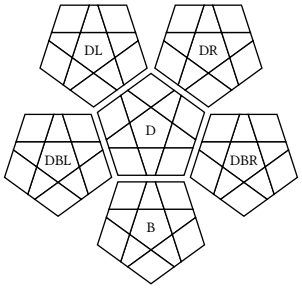


skewb

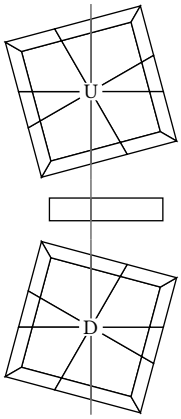


pyraminx





megaminx



square1

2.1 Notation

WCA notation, plus whole-puzzle rotations where the WCA has none. Spaces optional where unambiguous (`RUR'U'`). `(R U R' U')3` repeats a group, `(..)'` inverts it. Commutators and conjugates (except for Square-1) nest freely: `[A, B]` is `A B A' B'`; `[A: B]`, or `A: B` without brackets, is `A B A'`; so `F: [R, U]` and `[F: [R, U]]` both read `F R U R' U' F'`. Like `(..)`, a bracket can be repeated or inverted: `[R, U]3`, `[R, U]'`. Bad input fails with a `cubst:` message.

N×N cubes	
`R L U D F B`	face turns, clockwise seen from that face
`' 2 2'`	counter-clockwise, half turn
`Rw r 3Rw`	wide turns; leading number = layer count
`M E S`	slices, following `L`, `D`, `F`
`x y z`	whole-cube rotations about the `R`, `U`, `F` axes
skewb	
`R L U B`	120° clockwise about the DBR, DFL, UBL, DBL corners, seen from the corner; UFR never moves
`y`	whole-puzzle quarter turn, clockwise seen from above
pyraminx	
`U L R B`	120° clockwise about that vertex, seen from it; tip and layer
`u l r b`	tip only

pyraminx

`y z`	whole-puzzle 120° turns: `y` about the `U` vertex, clockwise from above; `z` about the `F` centre, clockwise from the front

megaminx

`U F R L BL BR DR DL DBR DBL B D`	face turns, one fifth; suffixes `' 2 2'`
`R++ R--`	all but L, two fifths, clockwise / counter-clockwise seen from the right
`D++ D--`	all but U, two fifths, seen from below

Square-1

`(x, y)`	top layer x·30°, bottom layer y·30°, clockwisely
`/`	right half 180°; a compile error when a corner blocks it

2.2 Color scheme

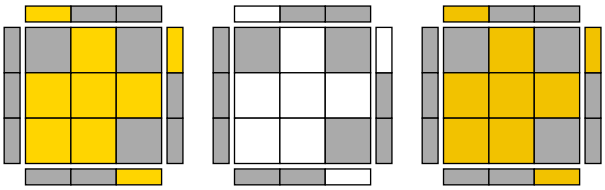
A sticker holds a color **name**, such as `"yellow"`, not a color. Two lookups turn a face into ink:

face → **scheme** `cube(scheme: ..)` → name → **palette** `draw(palette: ..)` → color

`U` → `"yellow"` → 

The scheme is part of the state: it decides which stickers end up where. The palette (Section 3.3.1) is part of the picture: it only changes the hues. Below, the same algorithm with the default scheme, with white on top (white is now "the top color"), and with another palette (same state, different yellow):

```
#let white-up = (..default-scheme,
  U: "white", D: "yellow", R: "red",
L: "orange")
#draw(case(sune), view: "oll")
#draw(case(sune, scheme: white-up),
view: "oll")
#draw(case(sune), view: "oll",
  palette: (..colors, yellow:
rgb("#f2c200")))
```



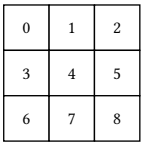
Every name in a scheme must exist in the palette. Views show faces by position, not by color; only the `oll`, `f2l` and `obl` masks read the scheme, to find the top (and bottom) color. The defaults:

N×N cubes, skewb	`(U: "yellow", D: "white", F: "green", B: "blue", R: "orange", L: "red")`
pyraminx	`(F: "green", L: "red", R: "blue", D: "yellow")`
megaminx	`(U: "white", F: "green", R: "red", BR: "blue", BL: "yellow", L: "purple", D: "grey", B: "lime", DBL: "orange", DL: "lightblue", DR: "beige", DBR: "pink")`
Square-1	`(U: "yellow", D: "white", F: "red", B: "orange", R: "green", L: "blue")`

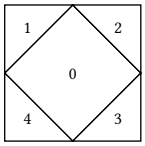
2.3 Reading a state

```
face(c, "U")             // stickers of a face: rows on a cube, else a flat array
sticker(c, "U", 0, 2)    // one color name, or none when hidden
is-solved(c)             // whether the cube is solved
```

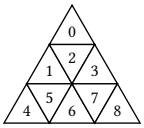
A **position** is a sticker's index on its face, as the `face` view draws it; `labels: true` writes the indices on the picture. N×N cubes also take `(row, col)` from the top left. Side faces are drawn with `U` at the top; `U` from above with `B` at the top; `D` from below with `F` at the top. On the pyraminx, `D` has the `B` vertex at the top. Megaminx faces always have a vertex at the top, so its side faces have `U` at the upper right. On the Square-1, `U` and `D` hold one sticker per piece, clockwise from the slice at the back, so their count changes with the state; the sides of the pieces and the equator are drawn but not indexed.



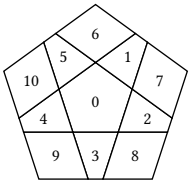
3x3, face U



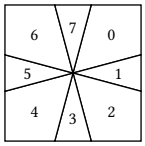
skewb, face U



pyraminx, face F



megaminx, face F



square1, face U

3 Drawing a state

```
draw(c, view: auto, mask: auto,
  sticker: 6mm, gap: 0pt, stroke: 0.5pt + black, radius: 0pt, body: none,
  palette: colors, hidden: auto,
  face: auto, top: auto, tip: auto,
  sides: auto, side-length: 0.35, labels: false,
  arrows: (), arrow-color: black, arrow-thickness: 1.6pt, arrow-head: 0.3,
  spacing: auto, options: (:))
```

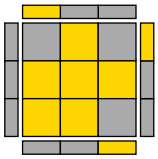
`view` picks the kind of picture and which stickers are hidden; `auto` is `full`, or `layers` on the Square-1. `mask` overrides the hiding. The rest is appearance and means the same in every view; a view ignores what it does not use, so one set of parameters serves all views.

3.1 Views

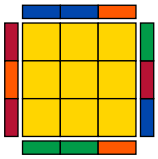
A view is a kind of picture plus a default **mask** (which stickers are greyed out). `face`, `net`, `full` and `layers` are the generic views; the others fix some arguments for a common diagram, so `view: "oll"` already hides everything but the top color. Asking a puzzle for a view it lacks is an error that lists the available ones.

view	picture	hidden by default	puzzles
`"face"`	one face straight on, chosen with `face:`	nothing	all
`"net"`	unfolded net	nothing	all
`"full"`	3D	nothing	N×N cubes, skewb, pyraminx, megaminx
`"f2l"`	`full` with a mask	every piece carrying the top color	N×N cubes

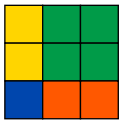
view	picture	hidden by default	puzzles
`"oll"`	`ll` with a mask	everything but the top color	N×N cubes
`"ll"`	`face` with `face: "U"`, `sides: true`	nothing	N×N cubes
`"tip"`	3D, looking down the vertex chosen with `tip:`	nothing	pyraminx
`"layers"`	both layers straight on	nothing	Square-1
`"obl"`	`layers` with a mask, caps only	everything but the `U` and `D` colors	Square-1
`"cs"`	`layers` with a mask, caps only	everything (shape only)	Square-1



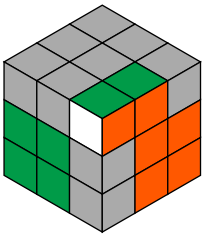
`"oll"`



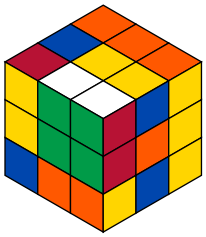
`"ll"`



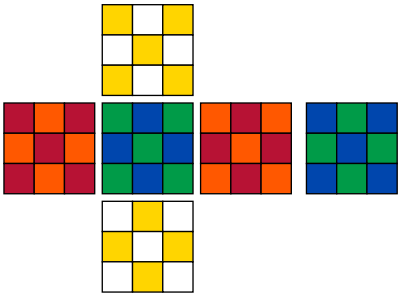
`"face", face: "F"`



`"f2l"`

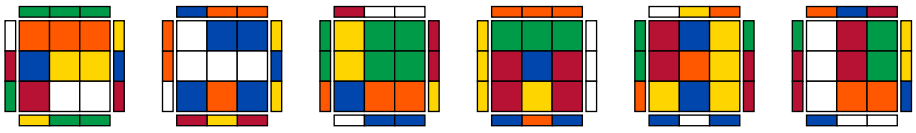


`"full"`



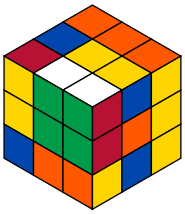
`"net"`

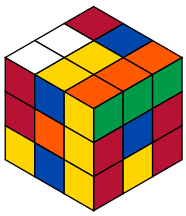
Straight on. One face head-on, oriented as in Section 2.3. `sides: true` adds every neighbouring sticker that touches the face as a strip outside the shared edge, so `U` with sides is the whole last layer seen from above. `sides: auto` is off for `face`, on for `ll` and `oll`.

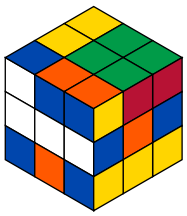


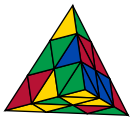
`face: "U"` `face: "D"` `face: "F"` `face: "B"` `face: "R"` `face: "L"`

3D. An orthographic view with `face` in front and `top` on top, `F` and `U` by default. The camera keeps its place relative to these two, so they must sit like `F` and `U`: adjacent faces, or on the pyraminx a face and one of its vertices (`top: auto` picks the first that fits `face`). N×N cubes and the skewb: from the corner between `face`, `top` and the face to their right. Pyraminx: from in front of `face`, slightly below, the `top` vertex up; `tip` straight down the vertex named by `tip:`, the usual last-layer picture. Megaminx: straight down `face` (Section 4.3).


default: `F, U`


`face: "R"`


`face: "D", top: "F"`


pyraminx `face: "D"`

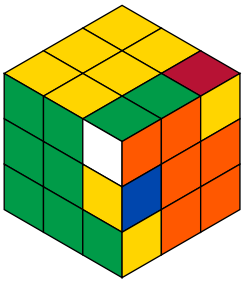
Net. Every face unfolded, as pictured in Section 2. The only view that shows every sticker.

Layers. The Square-1's two layers straight on: `U` from above, `D` from below, each with the sides of its pieces around it, the slice on the right marked by a grey line; side by side, or top to bottom with `options: (direction: "vertical")`. The equator only appears in `net`.

3.2 Mask

Leave `mask:` out for the view's default. `mask: none` shows every sticker; `mask: f` uses your own function from a state to a state. Hidden stickers are drawn in the `hidden` color.

```
#draw(case("U R U' R'"), view: "f2l", mask: none)
```



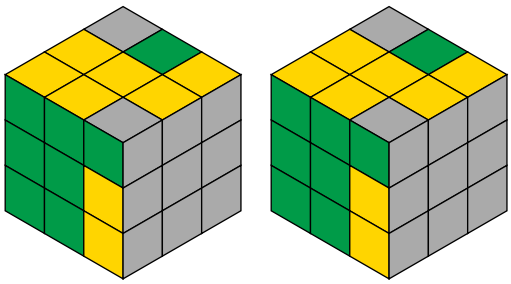
These functions return a state with some stickers hidden. They only ever hide, so they combine in any order, and work on every puzzle:

`keep-colors(c, "yellow")`	keep only these color name(s)
`hide-faces(c, ("D", "B"))`	hide whole faces
`hide-pieces(c, containing: "yellow")`	hide every piece that has a sticker of that color
`mask(c, info => ..)`	keep stickers for which the function is true; `info` has `face`, `index`, `color`, `piece`, plus `row`, `col` on N×N cubes

Pass one as `mask:` (as a function, so `draw` fills in the state) or apply it to the state first; the result is the same:

```
#draw(case("R U R' U'"), view: "full",
  mask: c => keep-colors(c, ("yellow",
"green")))

#draw(keep-colors(case("R U R' U'"),
("yellow", "green")),
  view: "full")
```



3.3 Appearance parameters

Lengths must be absolute (`mm`, `pt`, `cm`). Sizes are in **units**: one unit is the edge of a cube sticker, or of the megaminx centre pentagon (at the default `cut`); a skewb face and a pyraminx edge are 3 units.

parameter	default	meaning	used by
`sticker`	`6mm`	length of one unit	all
`gap`	`0pt`	space between stickers; `body` shows through it	all
`stroke`	`0.5pt + black`	sticker outline: one Typst stroke (thickness + color), a thickness, a color, a dictionary, or `none`	all
`radius`	`0pt`	corner radius of square stickers	all but 3D
`body`	`none`	color behind the stickers, seen through gaps and the margin around a face	all
`palette`	`colors`	color name ⟶ color (Section 3.3.1)	all
`hidden`	`auto`	color of masked stickers; `auto` = the palette's `hidden` entry	all
`face`	`auto`	`face` view: the face shown, `auto` = `U` (`F` on the pyraminx); `full`: the face in front, `auto` = `F`; not on the Square-1	face, full
`top`	`auto`	`full`: the face on top (pyraminx: the vertex); `auto` = `U`, or the first that fits `face`	full
`tip`	`auto`	vertex the pyraminx `tip` view looks down: `U`, `L`, `R` or `B`; `auto` = `U`	tip
`sides`	`auto`	draw the neighbours' stickers as strips; `auto` = off for `face`, `obl`, `cs`, on for `ll`, `oll`, `layers`	face, ll, oll, layers, obl, cs
`side-length`	`0.35`	strip thickness in units	face, ll, oll, layers, obl, cs, net
`labels`	`false`	`true` writes each sticker's index on it; `"faces"` writes the face names instead	face, ll, oll, layers, obl, cs, net
`arrows`	`()`	arrows between positions (Section 3.3.3)	face, ll, oll
`arrow-color`	`black`	default arrow color	face, ll, oll
`arrow-thickness`	`1.6pt`	default arrow line thickness	face, ll, oll
`arrow-head`	`0.3`	default arrow head length in units	face, ll, oll

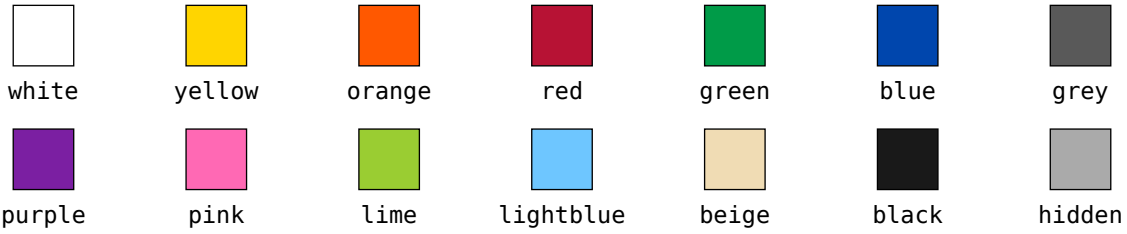
parameter	default	meaning	used by
`spacing`	`auto`	distance between faces; `auto` = a quarter unit (half a unit between layers)	net, layers, obl, cs
`options`	`(:)`	settings of one puzzle, listed below; other keys are errors	see below

Like `cube(options:)`, `draw(options:)` holds the settings that exist for one puzzle only:

puzzle	option	default	meaning
Square-1	`slices`	`auto`	mark the slice with a grey line; `auto` = on except for `face`
Square-1	`direction`	`"horizontal"`	layers side by side, or `"vertical"` for top to bottom
Square-1	`turn`	`false`	turn each layer so that its slice is vertical, as `net` always does

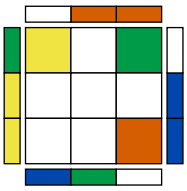
3.3.1 Palette

The palette maps color **names** to colors (the second lookup in Section 2.2). It belongs to the picture, not the state, so one state can be drawn with several palettes. The default `colors` has the six cube colors, five more for the megaminx, `black`, and `hidden` for masked stickers (`grey` is an ordinary color):



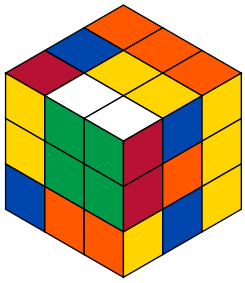
Spread `colors` to change some entries, for example for print:

```
#let print = (..colors,
  red: rgb("#d55e00"), orange: rgb("#f0e442"),
  yellow: rgb("#ffffff"), white: rgb("#999999"))
#draw(case(sune), view: "ll", palette: print)
```

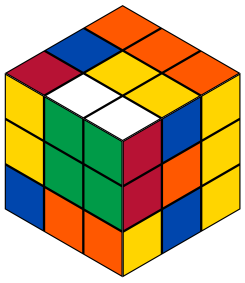


3.3.2 Outlines and gaps

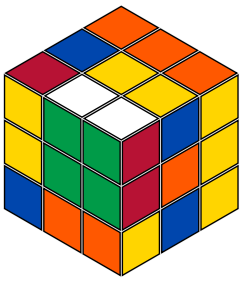
`stroke` is the sticker outline, one Typst stroke value (`0.5pt + black` is a single argument, as in `rect`). `gap` shrinks each sticker towards its centre by half the gap; the space shows `body`, which also fills the margin between a face and its strips. So stickers are separated by outlines, by gaps, or both:



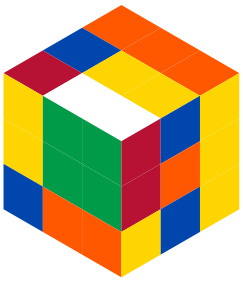
default
`stroke: 0.5pt + black`
`gap: 0pt`



gaps only
`stroke: none`
`gap: 1pt, body: black`

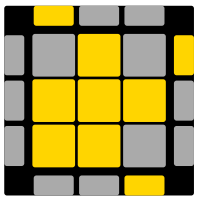


both
`gap: 1pt`
`body: luma(230)`



neither
`stroke: none`
`gap: 0pt`

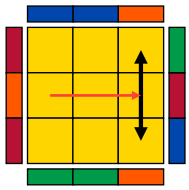
```
#draw(case(sune), view: "oll",
  stroke: none, gap: 1pt, body: black,
  radius: 1pt, side-length: 0.5)
```



3.3.3 Arrows

Arrows join positions (Section 2.3) on the shown face of any puzzle. Each is a pair `(from, to)` or a dictionary with `from`, `to` and optionally `double: true`, `color`, `thickness` and `head`, the last three overriding the `arrow-*` defaults for that arrow:

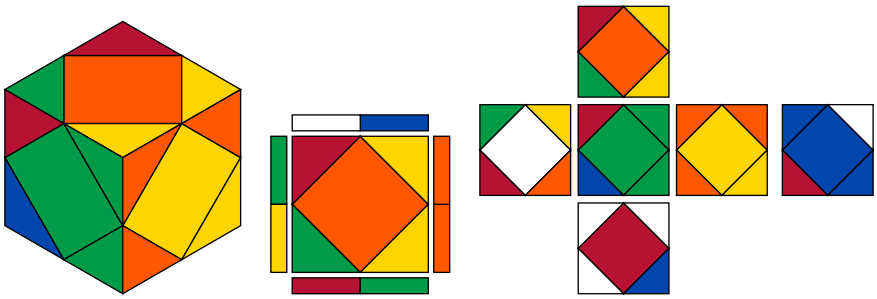
```
#draw(case(tperm), view: "ll", arrows: (
  (from: (0, 2), to: (2, 2), double: true),
  (from: (1, 0), to: (1, 2),
    color: red, thickness: 1pt, head: 0.2),
))
```



4 Other puzzles

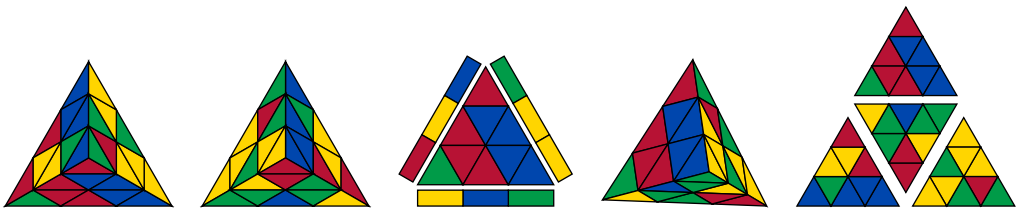
4.1 Skewb

```
#let c = cube(event: "skewb",
  scramble: "R L U B' R' L")
#draw(c, view: "full")
#draw(c, view: "face", face: "U", sides: true)
#draw(c, view: "net", sticker: 4mm)
```



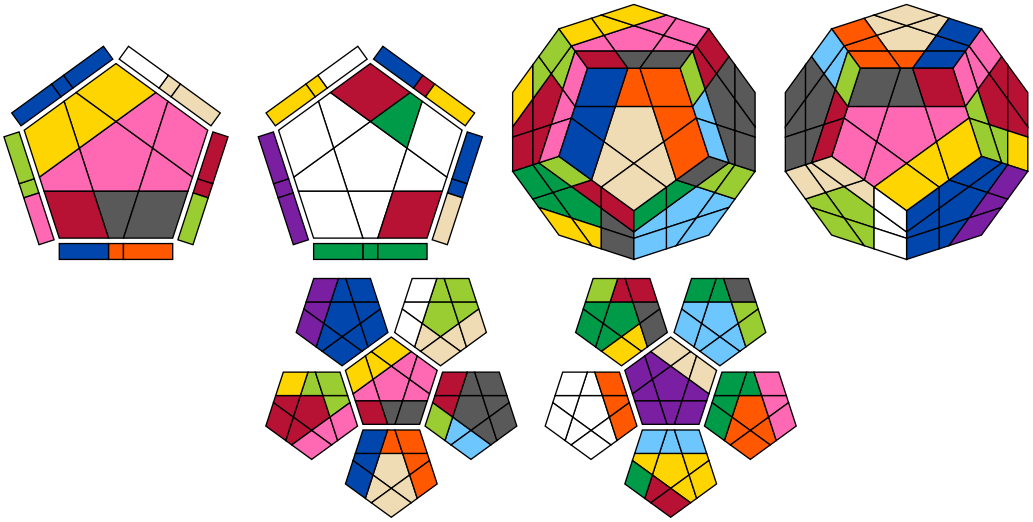
4.2 Pyraminx

```
#let c = cube(event: "pyraminx",
  scramble: "U L R' B u l'")
#draw(c, view: "tip")
#draw(c, view: "tip", tip: "B")
#draw(c, view: "face", sides: true)
#draw(c, view: "full")
#draw(c, view: "net", sticker: 4.5mm)
```



4.3 Megaminx

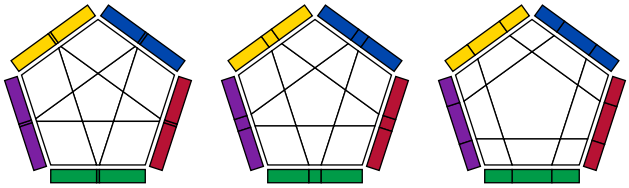
```
#let c = cube(event: "megaminx",
  scramble: "R++ D-- R++ D++ U F' BL2")
#draw(c, view: "face", sides: true)
#draw(case("R U R' U'", event: "megaminx"),
  view: "face", sides: true)
#draw(c, view: "full", sticker: 4mm)
#draw(c, view: "full", face: "U", sticker: 4mm)
#draw(c, view: "net", sticker: 3mm)
```



`full` looks straight down `face` with `top` up and shows it with its five neighbours: `F` by default, or `face: "U"` for the top half (then `F` is on top).

`cut` is the centre pentagon's inradius as a fraction of the face's: `0.4` by default, between `0.05` and `0.95`. Keep it above `0.31`, below which the edge stickers no longer reach the edge of the face. It changes the sticker shapes and the layer depth, so it is a `cube()` option, not a `draw()` one:

```
#for cut in (0.33, 0.4, 0.6) {
  draw(cube(event: "megaminx", options: (cut: cut)),
    view: "face", sides: true, sticker: 5mm)
}
```

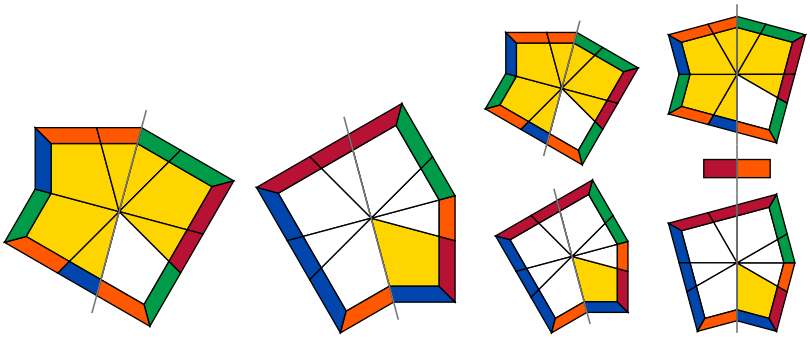


4.4 Square-1

Held the WCA way: yellow on top, white below, red in front, the slice through `F` and `B` with the smaller front part on the left. The shape follows the state.

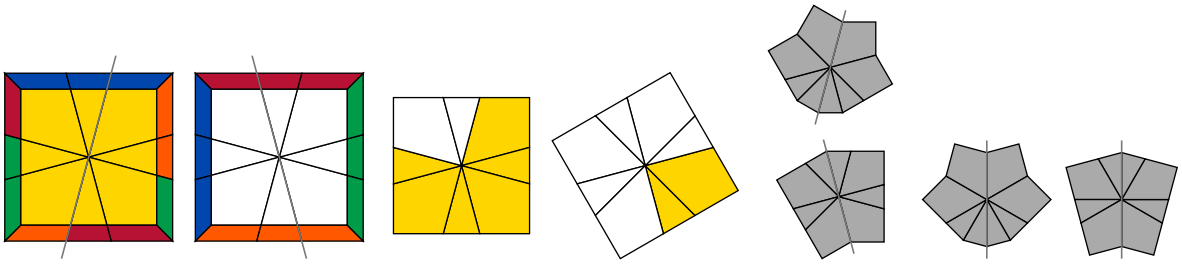
Views (Section 3.1). `layers`: `U` from above and `D` from below, each with the sides of its pieces, the slice on the right marked by a grey line. `net`: the layers turned so that the slice is vertical, with the front of the equator between them: two reds of different lengths when it is in place, a red and an orange of the same length when it is flipped. `obl` keeps only the `U` and `D` colors (the orientation diagram); `cs` hides every sticker (the cube-shape diagram); both show only the caps unless `sides: true`. `face:` and `top:` do not apply.

```
#let c = cube(event: "square1",
  scramble: "(0,-1)/ (3,0)/ (4,0)/ (6,0)/ (3,0)/")
#draw(c, view: "layers")
#draw(c, view: "layers", sticker: 4mm,
  options: (direction: "vertical"))
#draw(c, view: "net", sticker: 4mm)
```



Options (Section 3.3). `direction: "vertical"` stacks the layers, `slices: false` removes the line, `turn: true` turns each layer so that the line is vertical:

```
#let adj = "/ (3,-3) / (3,0) / (-3,0) / (0,3) / (-3,0) /"
#let sk = cube(event: "square1", scramble: "/ (3,0) / (1,2) /")  // scallop / kite
#draw(case(adj, event: "square1"), view: "layers")
#draw(cube(event: "square1", scramble: "(0,-1) / (-3,0) /"),
  view: "obl", options: (slices: false))
#draw(sk, view: "cs", sticker: 4mm, options: (direction: "vertical"))
#draw(sk, view: "cs", sticker: 4mm, options: (turn: true))
```

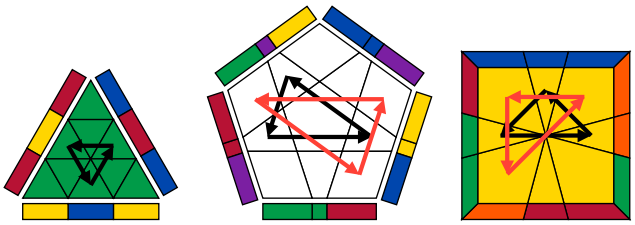


Moves. A `/` with a corner across the slice is a compile error naming the moves done so far. `is-solved` means cube shape with every piece in place, up to turning the layers.

4.5 Arrows on other puzzles

Arrows (Section 3.3.3) take the positions of Section 2.3. Below: an edge cycle on the pyraminx, the T perm on the megaminx, and the Square-1 algorithm above, which cycles three corner-edge blocks; corner cycles in red:

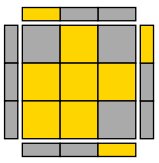
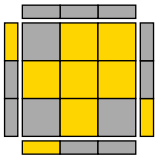
```
#let corner(a, b) = (from: a, to: b, color: red)
#draw(case("L' U L U R U R'", event: "pyraminx"),
  view: "face", sides: true, arrows: ((3, 1), (1, 6), (6, 3)))
#draw(case(tperm, event: "megaminx"), view: "face", sides: true,
  arrows: ((4, 2), (2, 5), (5, 4),
    corner(8, 7), corner(7, 10), corner(10, 8)))
#draw(case(adj, event: "square1"), view: "face", sides: true,
  arrows: ((5, 1), (1, 7), (7, 5),
    corner(4, 0), corner(0, 6), corner(6, 4)))
```



5 Recipes

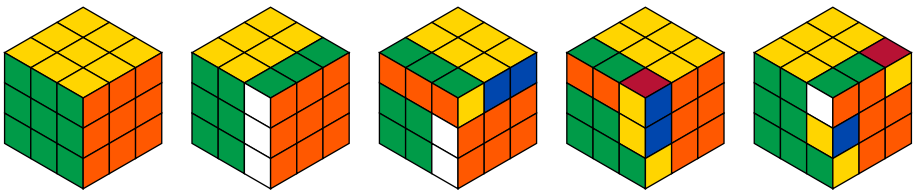
5.1 Algorithm sheet

```
#let algs = (
  ("Sune", "R U R' U R U2 R'"),
  ("Anti-Sune", "R U2 R' U' R U'
R'"),
)
#table(
  columns: 3, align: horizon,
  ..algs.map(((name, alg)) => (
    name, draw(case(alg), view:
"oll", sticker: 5mm), raw(alg)
  )).flatten(),
)
```

Sune		`R U R' U R U2 R'`
Anti-Sune		`R U2 R' U' R U' R'`

5.2 Step by step

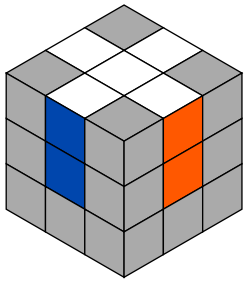
```
#let steps = ("R", "U", "R'", "U'")
#let states = range(steps.len() + 1)
  .map(k => cube(scramble: steps.slice(0, k).join(" ")))
#grid(columns: 5, gutter: 4mm,
  ..states.map(c => draw(c, view: "full", sticker: 4mm)))
```



5.3 Cross

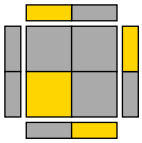
Only the white edges and the centres. `hide-pieces` hides exactly the pieces carrying white, so a sticker it turned into `none` is on one; a `row` or `col` of 1 rules out corners. `x2` puts white on top, where `full` can see it.

```
#let c = apply(cube(), "x2")
#let rest = hide-pieces(c, containing: "white")
#let cross = mask(c, info =>
  (info.row == 1 and info.col == 1)      // every centre ..
  or (rest.faces.at(info.face).at(info.index) == none
    and (info.row == 1 or info.col == 1))) // .. white edges
#draw(cross, view: "full")
```

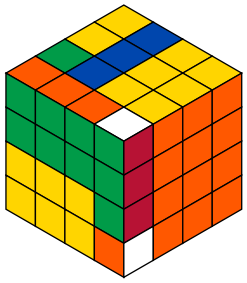


5.4 Other cube sizes

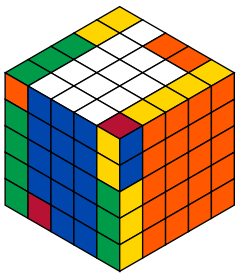
Every cube view works for any `"NxN"`. Wide moves take a layer count (`3Rw`); `M`, `E`, `S` turn all middle layers.



`"2x2", "oll"`



`"4x4", "Rw U 3Rw' F2"`



`"5x5", "3Rw U 3Rw' M2"`

6 Reference

function	**purpose**
`cube(event:, scramble:, inverted:, scheme:, options:)`	build a state
`case(alg, event:, scheme:, options:)`	the state an algorithm solves
`solved(event:, scheme:, options:)`	a solved state
`apply(c, alg)`	apply moves, returns a new state
`is-puzzle(x)`	whether a value is a state
`parse(alg, event:, options:)`	algorithm string ⟶ move array (`apply` accepts both)
`inverse(alg, event:, options:)`	the inverse, as a move array
`to-string(moves, event:, options:)`	move array ⟶ notation
`face(c, name)`, `sticker(c, name, ..pos)`, `is-solved(c)`	read a state (Section 2.3)
`keep-colors`, `hide-faces`, `hide-pieces`, `mask`	hide stickers (Section 3.2)
`draw(c, view:, mask:, ..)`	draw a state (Section 3)
`draw-face`, `draw-3d`, `draw-net`, `draw-layers`	the renderers behind `draw`; `draw-3d` also takes a `camera: (dir:, up:)`
`views`	the view table: kind, fixed arguments and default mask of each view
`colors`, `default-scheme`	the default palette and cube scheme
`puzzles`	the puzzle definitions, keyed `cube`, `skewb`, `pyraminx`, `megaminx`, `square1`